Cybersécurité avancée

Informatique — Licence 3 (L3)

Cours INF-L3-01

Institut Africain de Management (IAM) — Bamako, Mali
Bibliothèque Universitaire — Bronbah Tech

À propos de ce cours

Sécuriser en profondeur un système d'information : cryptographie moderne et certificats numériques, sécurité des applications web avec le référentiel OWASP Top 10, tests d'intrusion strictement encadrés par le droit, réponse aux incidents, investigation numérique et segmentation des réseaux — jusqu'à un projet d'audit complet mené sur un site de test autorisé.

Document pédagogique original créé pour les étudiants de l'IAM : leçons détaillées, exemples commentés, travaux dirigés et pratiques, exercices corrigés, QCM d'évaluation et vidéos recommandées. Aucun contenu n'est reproduit d'ouvrages existants.

Sommaire

• Introduction à la matière
• Objectifs d'apprentissage
• Leçon 1 — Cryptographie moderne : AES, RSA et certificats numériques
• Leçon 2 — Infrastructure à clés publiques et TLS : sécuriser les échanges web
• Leçon 3 — Sécurité des applications web : le référentiel OWASP Top 10
• Leçon 4 — Tests d'intrusion : méthodologie et cadre légal strict
• Leçon 5 — Réponse aux incidents et initiation à l'investigation numérique
• Leçon 6 — Sécurité des réseaux d'entreprise segmentés
• Leçon 7 — Projet : audit de sécurité d'un site de test autorisé
• Travaux dirigés (TD)
• Travaux pratiques (TP)
• Exercices corrigés supplémentaires
• QCM d'évaluation
• Résumé — les notions importantes
• Vidéos associées aux leçons
• Sources et références

Comment utiliser ce cours : lis chaque leçon dans l'ordre, refais les exemples à la main, tente chaque exercice AVANT de lire son corrigé, puis termine par le QCM pour vérifier ce que tu as retenu.

Introduction à la matière

Tu sais déjà te protéger au quotidien : mots de passe solides, mises à jour régulières, vigilance face aux messages suspects. En Licence 3, ce cours te fait passer de l'utilisateur prudent au professionnel qui protège les autres. Sécuriser un système d'information, c'est sécuriser une organisation entière : ses étudiants, ses clients, ses archives, sa réputation. Les attaquants, eux, ne choisissent jamais la porte la mieux gardée : ils cherchent la fenêtre oubliée — un composant jamais mis à jour, un mot de passe laissé par défaut, un réseau où chaque poste voit tous les autres. La défense doit donc être méthodique, organisée en couches, et capable de prouver ce qu'elle fait. C'est exactement le programme des sept leçons qui suivent, étudiées du point de vue de celui qui défend.

Le fil conducteur est simple : protéger les données quand elles circulent (cryptographie, certificats, protocole TLS), quand elles sont traitées (applications web corrigées selon le référentiel OWASP), quand elles sont hébergées (réseaux segmentés), et savoir réagir le jour où une protection cède (réponse aux incidents, investigation numérique). Un point ne changera jamais dans ce cours : la ligne rouge du droit. Tester, auditer, vérifier — oui, mille fois oui, mais uniquement sur ses propres systèmes ou avec une autorisation écrite explicite du propriétaire. C'est cette discipline qui distingue le professionnel de sécurité du délinquant, et elle conditionne la valeur de tout le reste. Le projet final te fera appliquer la démarche complète, de l'autorisation écrite au rapport d'audit, sur un site de test créé par ton équipe.

Ce cours suppose acquis les bases des réseaux et du web de Licence 1 et 2, l'hygiène numérique de première année et le panorama des menaces et protections de deuxième année. Il demande de la rigueur plus que des mathématiques avancées : les démonstrations cryptographiques sont exécutées en Python sous tes yeux, chaque correctif de code est montré en version « avant / après », et chaque procédure est datée et documentée comme elle le serait en entreprise. À la fin, tu disposeras d'une méthode complète et défendable — celle qu'attendent les employeurs, et celle que la loi autorise. Garde ce document comme une boîte à outils : les grilles, les plans et les modèles qu'il contient sont faits pour être réutilisés en stage comme en premier poste.

Objectifs d'apprentissage

À la fin de ce cours, tu seras capable de :

- Expliquer et mettre en œuvre le chiffrement hybride (AES + RSA) et le rôle du hachage SHA-256.
- Décrire une infrastructure à clés publiques : certificat X.509, chaîne de confiance et cycle de vie d'un certificat.
- Expliquer le fonctionnement de TLS 1.2 et 1.3 et les réglages défensifs d'un site web (HTTPS, HSTS, en-têtes de sécurité).
- Classer des constats de sécurité dans les catégories du référentiel OWASP Top 10 et proposer le correctif adapté à chacun.
- Présenter la méthodologie d'un test d'intrusion légal : autorisation écrite, périmètre, règles d'engagement et rapport.
- Appliquer les étapes d'une réponse aux incidents et préserver des preuves numériques de façon documentée.
- Concevoir un réseau d'entreprise segmenté en zones, avec des flux autorisés minimaux, explicites et journalisés.

Leçon 1 — Cryptographie moderne : AES, RSA et certificats numériques

La cryptographie moderne fournit quatre services que tout le cours va utiliser : la confidentialité (seuls les destinataires légitimes lisent le message), l'intégrité (toute modification est détectable), l'authenticité (on sait qui a émis le message) et la non-répudiation (l'émetteur ne peut pas nier son envoi signé). Elle repose sur deux familles d'algorithmes. Le chiffrement symétrique utilise une seule clé, partagée, pour chiffrer et déchiffrer : il est très rapide, mais il pose un problème redoutable — comment transmettre cette clé secrète à un correspondant que l'on n'a jamais rencontré, sans qu'elle soit interceptée ? Le chiffrement asymétrique résout ce problème avec des paires de clés, au prix d'une lenteur mille fois supérieure. Toute l'architecture de la sécurité moderne consiste à combiner intelligemment les deux familles.

L'algorithme symétrique de référence est AES (Advanced Encryption Standard), standardisé par le NIST en 2001 dans la publication FIPS 197. AES chiffre les données par blocs de 128 bits, avec des clés de 128, 192 ou 256 bits : plus la clé est longue, plus la recherche exhaustive est hors de portée — avec 256 bits, elle dépasse toute

capacité de calcul imaginable. Deux précisions comptent plus que la taille de clé. D'abord le mode d'opération : les modes authentifiés comme AES-GCM protègent à la fois la confidentialité et l'intégrité, tandis que le vieux mode ECB, qui chiffre chaque bloc indépendamment, laisse apparaître les motifs des données et ne doit plus être utilisé. Ensuite la gestion du nonce : en mode GCM, le nonce (nombre à usage unique) ne doit jamais être réutilisé avec la même clé. AES protège aujourd'hui les disques chiffrés, les sauvegardes, les messageries et l'essentiel des connexions TLS.

RSA, publié en 1977 par Rivest, Shamir et Adleman, est l'algorithme asymétrique historique. Sa sécurité repose sur un déséquilibre : multiplier deux très grands nombres premiers est facile, mais retrouver ces deux facteurs à partir du seul produit est, en pratique, hors de portée. Chaque utilisateur génère une paire : la clé publique, que l'on diffuse, et la clé privée, que l'on ne partage jamais. Ce que la clé publique chiffre, seule la clé privée le déchiffre ; et inversement, une signature produite avec la clé privée se vérifie avec la clé publique. Les clés RSA de 1 024 bits sont considérées comme insuffisantes : on utilise au minimum 2 048 bits, et 3 072 bits pour les secrets à long terme. Trop lent pour chiffrer des volumes, RSA sert à deux choses : transporter en sécurité une clé symétrique fraîche — c'est le chiffrement hybride — et signer. Le hachage, lui, ne chiffre rien : SHA-256 transforme n'importe quelle donnée en une empreinte de 256 bits, irréversible et quasi unique, qui sert à vérifier l'intégrité et à construire les signatures. Enfin, un certificat numérique lie une identité à une clé publique : c'est le pont entre la cryptographie et la confiance, étudié à la leçon suivante.

L'essentiel
AES (clé de 128/192/256 bits, mode authentifié GCM) chiffre vite les données ; RSA (paire de clés, 2 048 bits minimum) transporte les clés et signe ; SHA-256 produit des empreintes d'intégrité ; le certificat numérique lie une identité à une clé publique. Le chiffrement hybride combine les trois.

Exemple — Chiffrement hybride AES-256-GCM + RSA-2048 exécuté en Python (bibliothèque cryptography)

```
# Démonstration DÉFENSIVE : chiffrement hybride AES-256-GCM + RSA-2048
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
import hashlib, os
message = "Relevé de notes du semestre 5 — confidentiel"
# 1) Le serveur génère sa paire de clés RSA de 2048 bits
cle_privee = rsa.generate_private_key(public_exponent=65537,
key_size=2048)
cle_publique = cle_privee.public_key()

# 2) Le client génère une clé AES de 256 bits et un nonce unique
cle_aes = os.urandom(32) # 32 octets = 256 bits
nonce = os.urandom(12) # jamais réutilisé avec la même clé

# 3) Chiffrement des données avec AES-256-GCM (mode authentifié)
texte_chiffre = AESGCM(cle_aes).encrypt(nonce, message.encode(), None)
# 4) La clé AES est chiffrée avec la clé PUBLIQUE RSA (OAEP + SHA-256)
cle_chiffree = cle_publique.encrypt(
cle_aes,
padding.OAEP(mgf=padding.MGF1(hashes.SHA256()),
algorithm=hashes.SHA256(), label=None))

# 5) Le serveur déchiffre la clé AES (clé PRIVÉE), puis les données
cle_recue = cle_privee.decrypt(
cle_chiffree,
padding.OAEP(mgf=padding.MGF1(hashes.SHA256()),
algorithm=hashes.SHA256(), label=None))
texte_clair = AESGCM(cle_recue).decrypt(nonce, texte_chiffre, None)

# 6) Empreinte SHA-256 du document (contrôle d'intégrité)
empreinte = hashlib.sha256(message.encode()).hexdigest()

print("Texte chiffré AES-GCM :", len(texte_chiffre), "octets")
print("Clé AES chiffrée par RSA :", len(cle_chiffree), "octets")
print("Clé retrouvée identique :", cle_recue == cle_aes)
print("Message déchiffré :", texte_clair.decode())
print("Empreinte SHA-256 :", empreinte)
# --- Sortie réelle obtenue à l'exécution de cette démonstration :
# Texte chiffré : 63 octets ; clé chiffrée par RSA : 256 octets
# Clé retrouvée identique : True ; message déchiffré identique
# Empreinte : d6f266028b4b7a86f3b532be875403e1fdccc...
# (les valeurs chiffrées changent à chaque exécution, car les
# clés et le nonce sont tirés au hasard ; longueurs stables)
```

À retenir

• Symétrique = rapide mais exige un secret partagé ; asymétrique = paires de clés, lent, réservé aux clés et aux signatures.

• AES : blocs de 128 bits, clés de 128/192/256 bits ; toujours un mode authentifié (GCM), jamais ECB, jamais un nonce réutilisé.

• Hacher n'est pas chiffrer : SHA-256 donne une empreinte irréversible de 256 bits pour vérifier l'intégrité.

Exercice

Le service informatique de l'université doit protéger trois choses : (a) l'archive complète des relevés de notes, plusieurs gigaoctets stockés sur un serveur ; (b) l'envoi de la clé de protection de cette archive à un partenaire extérieur ; (c) la possibilité de prouver, dans un an, qu'un relevé précis n'a pas été modifié entre-temps. Pour chaque besoin, choisis l'outil principal (AES, RSA ou SHA-256), précise le paramètre clé (taille ou mode) et justifie en une ou deux phrases.

✔ Corrigé
(a) AES, en mode authentifié GCM avec une clé de 256 bits : seul le chiffrement symétrique a le débit nécessaire pour plusieurs gigaoctets, et GCM détectera toute modification de l'archive. (b) RSA (2 048 bits minimum, remplissage OAEP) : on chiffre la clé AES — quelques dizaines d'octets seulement — avec la clé publique du partenaire ; seul lui pourra la déchiffrer avec sa clé privée. C'est le principe du chiffrement hybride de la démonstration : le texte chiffré RSA fait exactement la taille de la clé RSA, 256 octets pour 2 048 bits. (c) SHA-256 : on calcule et on conserve l'empreinte du relevé au moment de l'archivage ; toute modification ultérieure, même d'un seul caractère, produira une empreinte totalement différente, comme on le voit dans la démonstration où l'empreinte est stable et reproductible à l'identique.

Leçon 2 — Infrastructure à clés publiques et TLS : sécuriser les échanges web

À quoi sert la clé publique d'un site que l'on visite pour la première fois ? Rien ne prouve qu'elle appartient bien à ce site : n'importe qui pourrait présenter la sienne. L'infrastructure à clés publiques (PKI, Public Key Infrastructure) résout ce problème d'identité. Au sommet, des autorités de certification (AC) racines, dont les certificats auto-signés sont intégrés aux navigateurs et aux systèmes d'exploitation : ce sont les racines de confiance. Pour limiter les risques, les racines signent rarement les certificats des sites directement : elles délèguent à des autorités intermédiaires, qui émettent les certificats des serveurs. Le certificat, au format X.509, contient l'identité du titulaire (nom de domaine, champ SAN — Subject Alternative Name), sa clé publique, l'émetteur, les dates de validité, les usages autorisés et la signature de l'AC émettrice. Quand ton navigateur reçoit un certificat, il remonte cette chaîne de signatures jusqu'à une racine qu'il connaît, vérifie chaque signature, la correspondance du nom de domaine et la période de validité : si un seul maillon échoue, il affiche une alerte — qu'il ne faut jamais ignorer sur un site réel.

Un certificat a un cycle de vie qu'il faut administrer sérieusement. D'abord la demande : le serveur génère sa paire de clés, conserve précieusement la clé privée et envoie une demande de signature (CSR) contenant la clé publique. L'autorité vérifie ensuite le contrôle du domaine (validation DV, automatisée), ou l'identité de l'organisation (OV, et EV pour les contrôles renforcés). Le certificat émis a une durée limitée — 90 jours chez l'autorité gratuite Let's Encrypt, qui a généralisé l'automatisation via le protocole ACME : le renouvellement devient une tâche de machine, pas un souvenir à ne pas oublier. Si la clé privée est compromise ou le certificat émis par erreur, il faut le révoquer : les mécanismes CRL (listes de révocation) et OCSP (interrogation en ligne) permettent aux navigateurs d'apprendre qu'un certificat ne doit plus être accepté

avant son expiration. La plupart des incidents « certificat » des entreprises sont banals : un renouvellement oublié, et le site devient inaccessible ou inquiétant du jour au lendemain.

TLS (Transport Layer Security, RFC 8446 pour la version 1.3) est le protocole qui met la PKI au travail à chaque connexion HTTPS. La poignée de main TLS 1.3 tient en un aller-retour : le navigateur propose ses paramètres, le serveur présente son certificat et prouve qu'il détient la clé privée correspondante en signant les échanges ; simultanément, les deux parties réalisent un échange de clés éphémères Diffie-Hellman (ECDHE) qui fabrique un secret de session tout neuf. Éphémère est le mot clé : même si la clé privée du serveur est volée plus tard, les conversations passées restent illisibles — c'est la confidentialité persistante (perfect forward secrecy). Ensuite, tout le trafic est chiffré en symétrique rapide, AES-GCM ou ChaCha20-Poly1305. TLS 1.2 reste présent dans beaucoup de parcs ; en revanche SSL et TLS 1.0/1.1 sont abandonnés et doivent être désactivés. Côté défense, on complète par le forçage de HTTPS et l'en-tête HSTS, qui ordonne au navigateur de ne plus jamais contacter le site en clair.

L'essentiel
La PKI transforme des clés publiques en identités vérifiables : AC racine de confiance, intermédiaires, certificat X.509 signé. TLS 1.3 authentifie le serveur par son certificat, crée un secret de session éphémère (ECDHE, confidentialité persistante) puis chiffre en AES-GCM. Un certificat expiré, auto-signé ou au mauvais nom doit toujours alerter.

Exemple — Chaîne de certificats d'un site fictif et poignée de main TLS 1.3

```
CHAÎNE DE CONFIANCE du site https://biblio.iam-edu.example
==========================================================
[1] Certificat du SERVEUR (présenté à chaque connexion)
Sujet (SAN) : biblio.iam-edu.example
Clé publique : ECDSA P-256 (du serveur)
Émetteur : AC Intermédiaire Enseignement
Validité : 90 jours, renouvellement automatique
Signature : de l'AC intermédiaire -> VÉRIFIÉE
|
v
[2] Certificat de l'AC INTERMÉDIAIRE
Sujet : AC Intermédiaire Enseignement
Émetteur : AC Racine Enseignement
Signature : de l'AC racine -> VÉRIFIÉE
|
v
[3] Certificat de l'AC RACINE (auto-signé)
Présent dans le magasin de confiance du navigateur
-> POINT D'ARRÊT : la chaîne est digne de confiance

POIGNÉE DE MAIN TLS 1.3 (un seul aller-retour)
Navigateur -> ClientHello : versions, suites, part ECDHE
Serveur -> ServerHello + certificat + signature
Navigateur : vérifie chaîne, nom, dates, révocation
Les deux : dérivent le secret de session éphémère
Ensuite : HTTP chiffré en AES-256-GCM (clés de session)
-> Même si la clé du serveur fuit demain, les échanges
d'hier restent illisibles (confidentialité persistante)
```

À retenir

- Le navigateur vérifie quatre choses : la chaîne de signatures, le nom de domaine, les dates de validité et la non-révocation.
- Racine de confiance préinstallée, intermédiaires déléguées : une racine compromise serait catastrophique, on l'expose donc le moins possible.
- TLS 1.3 = authentification par certificat + échange éphémère ECDHE + chiffrement symétrique de session ; activer HTTPS partout et HSTS.

Exercice

Trois utilisateurs signalent trois alertes différentes sur des sites qu'ils visitent : (1) « certificat expiré depuis hier » sur le site de la banque de l'université ; (2) « certificat auto-signé » sur le serveur de test du laboratoire, installé par l'enseignant et joignable uniquement depuis le réseau interne ; (3) « le certificat est émis pour un autre nom de domaine » sur une copie du site de la bibliothèque hébergée à une adresse inconnue. Pour chaque cas : quelle vérification du navigateur a échoué, le risque est-il acceptable, et quelle conduite recommandes-tu ?

✔ Corrigé
(1) La vérification des dates a échoué. Sur un site bancaire réel, le risque n'est pas acceptable : on ne clique jamais « continuer quand même » ; on prévient le service informatique, qui doit renouveler d'urgence — un certificat expiré est d'abord un défaut d'administration. (2) C'est la chaîne de confiance qui « échoue », volontairement : un certificat auto-signé est légitime dans un laboratoire fermé, à condition que l'enseignant ait communiqué son empreinte par un canal sûr et que le serveur ne soit jamais exposé sur Internet. Le risque est acceptable dans ce périmètre précis, et uniquement dans celui-ci. (3) La vérification du nom (SAN) a échoué : le certificat présenté ne couvre pas le nom visité. Sur une copie à adresse inconnue, c'est le signal typique d'un site d'hameçonnage : on ferme la page, on ne saisit aucun identifiant, et on signale l'adresse au service informatique. Retenir la règle générale : une alerte de certificat n'est jamais un détail technique à contourner, c'est une vérification de sécurité qui a échoué.

Leçon 3 — Sécurité des applications web : le référentiel OWASP Top 10

L'OWASP (Open Web Application Security Project) est une fondation ouverte qui publie des référentiels gratuits pour la sécurité des applications. Son Top 10 classe, à partir de données collectées sur des milliers d'applications réelles, les dix catégories de risques les plus critiques. L'édition 2021, la plus documentée et encore la référence des cours et des audits, s'ouvre par A01 — le contrôle d'accès défaillant, suivi notamment des échecs cryptographiques (A02), des injections (A03), de la conception non sécurisée (A04), des mauvaises configurations (A05), des composants vulnérables ou périmés (A06), des échecs d'identification et d'authentification (A07), des échecs d'intégrité des logiciels et des données (A08), des défauts de journalisation et de surveillance (A09) et des requêtes forgées côté serveur, SSRF (A10). Une édition 2025 a depuis été publiée ; les catégories bougent, mais les réflexes de défense de cette leçon restent les mêmes. Attention à l'esprit du classement : ce sont des familles de risques, pas des recettes d'attaque, et le Top 10 est un plancher de sécurité — pas un plafond.

Regardons les grandes familles du côté du défenseur. Le contrôle d'accès défaillant (A01) se corrige par une règle simple : le serveur vérifie, pour CHAQUE requête, que l'utilisateur connecté a le droit d'accéder à la ressource demandée — jamais en se fiant à un menu caché ou à un identifiant deviné dans l'adresse. Les injections (A03) naissent quand une donnée saisie par l'utilisateur est traitée comme du code par la base de données ou la page : le correctif fondateur est la séparation stricte entre le code et les données, par les requêtes paramétrées côté base et l'encodage des sorties côté affichage (c'est aussi la parade contre les scripts injectés dans les pages, dits XSS). Les échecs cryptographiques (A02) couvrent les données sensibles exposées : mots de passe stockés sans protection adaptée, échanges non chiffrés, secrets écrits dans le code. Un mot de passe ne se « chiffre » pas de manière réversible : il se hache avec une fonction lente et salée conçue pour cela (Argon2, bcrypt), afin que même la base volée résiste aux attaques par essais massifs.

Les autres familles sont tout aussi concrètes. La conception non sécurisée (A04) rappelle que la sécurité se décide dès la conception : un processus de réinitialisation de mot de passe mal pensé ne sera jamais sauvé par un beau code. La mauvaise configuration (A05) rassemble les comptes par défaut jamais changés, les messages d'erreur qui racontent toute la technique du serveur, les services inutiles restés actifs. Les composants vulnérables (A06) imposent un inventaire vivant des bibliothèques et leurs mises à jour : la plupart des compromissions exploitent des failles connues et déjà corrigées ailleurs. Les échecs d'authentification (A07) se traitent par l'authentification multifacteur, la limitation des tentatives et des sessions bien gérées. Sans journalisation ni surveillance (A09), une intrusion peut durer des mois sans être vue. La méthode de travail qui découle du Top 10 tient en une phrase : pour chaque famille, connaître la cause, appliquer le correctif de conception, puis vérifier qu'il est en place — c'est exactement le travail de l'audit de la leçon 7.

L'essentiel
Le Top 10 OWASP (édition 2021) classe les dix familles de risques web les plus critiques. Le correctif est toujours du côté du défenseur : contrôler les accès côté serveur à chaque requête, séparer code et données (requêtes paramétrées, encodage des sorties), hacher les mots de passe avec une fonction lente et salée, tenir les composants à jour et journaliser.

Exemple — Correctif AVANT / APRÈS : requête paramétrée (injection) et encodage de sortie — version APRÈS exécutée

```
# ------------------------------------------------------------
# AVANT (À NE PAS FAIRE) : la saisie de l'utilisateur est
# collée directement dans le texte de la requête SQL.
# La base ne distingue plus le code des données reçues :
# requete = "SELECT * FROM etudiants WHERE nom = '"
# requete = requete + saisie + "'"
# con.execute(requete)
# ------------------------------------------------------------
# APRÈS (CORRIGÉ) : requête paramétrée. Le « ? » fait passer
# la saisie comme une DONNÉE, jamais comme du code SQL.
import sqlite3

con = sqlite3.connect(":memory:")
con.execute("CREATE TABLE etudiants (id INTEGER, nom TEXT, filiere TEXT)")
con.executemany("INSERT INTO etudiants VALUES (?, ?, ?)",
[(1, "Awa Ndiaye", "Informatique"),
(2, "Moussa Diarra", "Informatique"),
(3, "Léa Martin", "Droit")])

def rechercher_etudiant(nom_saisi):
curseur = con.execute(
"SELECT id, nom, filiere FROM etudiants WHERE nom = ?",
(nom_saisi,))
return curseur.fetchall()

print("Recherche normale :", rechercher_etudiant("Awa Ndiaye"))
# Une saisie hostile est traitée comme un simple nom à chercher :
print("Saisie hostile :", rechercher_etudiant("' OR '1'='1"))
print("Lignes restantes :",
con.execute("SELECT COUNT(*) FROM etudiants").fetchone()[0])
# Sorties réelles : [(1, 'Awa Ndiaye', 'Informatique')] ; [] ; 3
#
# Deuxième correctif : ENCODAGE DE SORTIE avant affichage HTML
# (parade contre les scripts injectés dans les pages, XSS).
import html
nom_recu = "<script>alert('x')</script>" # donnée d'un formulaire
print("Affichage protégé :", html.escape(nom_recu))
# Sortie réelle : &lt;script&gt;alert(&#x27;x&#x27;)&lt;/script&gt;
# -> le navigateur affiche un texte inerte, il n'exécute rien.
```

À retenir

- A01 à A10 sont des familles de risques : pour chacune, on retient la cause, le correctif et la vérification — jamais une recette d'exploitation.
- Injection : la donnée ne doit jamais devenir du code — requêtes paramétrées partout, sans exception, et validation des entrées en complément.
- Mot de passe = hachage lent et salé (Argon2, bcrypt) ; jamais de texte clair, jamais de chiffrement réversible, jamais un simple SHA-256 rapide.

Exercice

Quatre constats relevés sur le site de test du laboratoire sont à classer : (1) en modifiant le numéro d'étudiant dans l'adresse de sa page de relevé, un étudiant connecté peut afficher le relevé d'un camarade ; (2) la page « mot de passe oublié »

envoie le mot de passe actuel par courriel ; (3) la bibliothèque graphique du site n'a pas été mise à jour depuis trois ans alors qu'un correctif de sécurité existe ; (4) le serveur affiche, en cas d'erreur, le texte complet des requêtes SQL et les chemins des fichiers. Pour chaque constat : catégorie OWASP Top 10 (2021), cause en une phrase, correctif principal.

✔ Corrigé
(1) A01 — contrôle d'accès défaillant : le serveur sert une ressource sur la seule foi de l'identifiant présent dans l'adresse, sans vérifier que le relevé appartient à l'étudiant connecté. Correctif : contrôle d'autorisation côté serveur à chaque requête (la ressource demandée doit appartenir à l'utilisateur ou à son rôle). (2) A02 — échecs cryptographiques, avec A07 en arrière-plan : l'application peut restituer le mot de passe, donc elle le stocke de façon réversible ou en clair. Correctif : stocker uniquement un hachage lent et salé (Argon2/bcrypt) et remplacer l'envoi par un lien de réinitialisation à usage unique et à durée courte. (3) A06 — composants vulnérables et périmés : la faille est connue et corrigée en amont, seul le retard de mise à jour expose le site. Correctif : inventaire des composants, veille sur leurs correctifs, procédure de mise à jour régulière et testée. (4) A05 — mauvaise configuration de sécurité : le mode « débogage » est resté actif en production et offre à tous le plan détaillé du système. Correctif : désactiver l'affichage d'erreurs détaillé côté public, journaliser les erreurs côté serveur, et afficher un message générique à l'utilisateur.

Leçon 4 — Tests d'intrusion : méthodologie et cadre légal strict

Un test d'intrusion (ou pentest) est une vérification de sécurité commandée et autorisée par le propriétaire d'un système, pour mesurer la résistance réelle de ses défenses avant qu'un attaquant véritable ne s'en charge. Trois conditions le rendent légitime, et elles ne sont pas négociables. Premièrement, une autorisation écrite et préalable, signée par une personne ayant le pouvoir de l'accorder : elle nomme le système concerné, les testeurs et la période. Deuxièmement, un périmètre précis : quelles adresses, quelles applications, quels comptes de test — tout ce qui n'est pas listé est interdit, y compris les systèmes des prestataires et des voisins d'hébergement. Troisièmement, des règles d'engagement : horaires autorisés, actions proscrites (par exemple tout ce qui pourrait interrompre le service ou altérer des données réelles), procédure d'arrêt d'urgence et contacts joignables jour et nuit. Sans ces trois éléments réunis, il n'y a pas de test d'intrusion : il y a une intrusion, tout court. Le test d'intrusion se distingue du simple scan de vulnérabilités, automatisé et sans validation, et de l'audit organisationnel, qui examine les procédures autant que la technique.

La méthodologie professionnelle suit des phases stables, décrites ici au niveau de la méthode — c'est elle que l'on attend d'un futur professionnel, pas des gestes d'exploitation. Phase 1, cadrage : contrat, autorisation, périmètre, règles d'engagement, et définition du succès (un rapport utile, pas un trophée). Phase 2, collecte d'informations sur le périmètre autorisé : inventaire des actifs exposés, des services publiés et de leurs versions — le même inventaire que le défenseur devrait

tenir en permanence. Phase 3, analyse des vulnérabilités : confrontation de cet inventaire aux bases de failles connues et examen des configurations, avec des outils d'analyse autorisés par le mandat. Phase 4, validation prudente : pour les failles critiques seulement, et toujours dans le périmètre, on vérifie de manière non destructive que le risque est réel, afin d'éviter les faux positifs dans le rapport ; dès qu'une preuve suffisante est obtenue, on s'arrête — on ne va jamais « plus loin pour voir ». Phase 5, rapport et remédiation : constats classés par criticité, preuves, impact compréhensible par un décideur et correctifs recommandés. Phase 6, contre-audit : après correction, on vérifie que les failles ont disparu sans en ouvrir d'autres. La valeur du test se mesure aux correctifs qu'il déclenche.

Le cadre légal est la frontière entre le métier et le délit. En droit, accéder ou se maintenir dans un système informatique sans l'autorisation de son propriétaire est un accès frauduleux, puni par la loi même sans dégât et même « pour vérifier » : en France, ce sont les articles 323-1 et suivants du Code pénal, issus de la loi dite Godfrain ; le Mali réprime également la cybercriminalité par une législation dédiée, et la plupart des pays ont des textes équivalents. L'autorisation du seul utilisateur d'un poste ne suffit pas si le système appartient à son employeur ou à son université. Pendant un test autorisé, les données rencontrées restent confidentielles : on n'en copie que le strict nécessaire à la preuve, on les protège, et le rapport lui-même devient un document sensible — il décrit les faiblesses de l'organisation. Pour s'entraîner légalement, les voies ne manquent pas : ses propres machines et machines virtuelles de laboratoire, les environnements de test fournis par l'école, les plateformes d'entraînement conçues pour cet usage et les programmes publics de divulgation encadrée lorsqu'ils existent. La règle d'or tient en une ligne : pas d'autorisation écrite, pas de test.

L'essentiel
Un test d'intrusion légitime repose sur trois piliers : autorisation écrite préalable du propriétaire, périmètre précis, règles d'engagement (limites, horaires, arrêt d'urgence). Sa méthodologie va du cadrage au contre-audit, en validation non destructive. Sans autorisation, tester un système tiers est un accès frauduleux puni par la loi — même sans dégât et même « pour aider ».

Exemple — Fiche d'autorisation et de périmètre (modèle) et phases du test

```
FICHE D'AUTORISATION DE TEST D'INTRUSION (à signer AVANT)
=========================================================
Commanditaire : Direction de l'école / du service _________
(nom et fonction du signataire, seul habilité à autoriser)
Testeurs : noms + entreprise/équipe ____________________
Périmètre : - application : site de test du laboratoire
- adresse locale : 192.168.56.20 uniquement
- comptes de test : testeur1, testeur2
- EXCLUS : réseau de production, postes
administratifs, hébergeur, autres sites
Fenêtre : du __/__/____ au __/__/____, 8 h - 18 h
Interdictions : interruption de service, suppression ou
altération de données, accès aux données
personnelles réelles (données fictives OK)
Arrêt : contact d'urgence joignable : _____________
mot de passe d'arrêt convenu : _____________
Livrables : rapport de constats + réunion de restitution
Signatures : commanditaire ___________ testeur _________

PHASES DU TEST (méthodologie)
1. Cadrage : contrat, autorisation, périmètre, règles
2. Inventaire des actifs exposés dans le périmètre
3. Analyse des vulnérabilités (failles connues, réglages)
4. Validation prudente et NON destructive des critiques
5. Rapport : constats, preuves, correctifs priorisés
6. Contre-audit après correction
-> Aucune phase n'autorise de sortir du périmètre signé.
```

À retenir

- Autorisation écrite préalable + périmètre + règles d'engagement : si l'un des trois manque, on ne commence pas.
- La validation est non destructive et minimale : dès que la preuve du risque suffit, on s'arrête et on documente.
- Le produit d'un test est un rapport de correctifs, protégé comme un document sensible — pas une démonstration d'exploits.

Exercice

Scénario : en rentrant chez lui, un étudiant de la classe remarque que la page d'administration du site d'une boutique de son quartier est accessible depuis Internet, protégée seulement par l'identifiant « admin » visible dans la page. Il envisage de « vérifier si le mot de passe est resté celui par défaut, juste pour prévenir ensuite la boutique ». Explique pourquoi il ne doit surtout pas le faire, ce qu'il peut faire à la place, et liste les cinq champs indispensables d'une éventuelle demande d'autorisation si la boutique souhaite ensuite un vrai test.

✔ Corrigé
Il ne doit pas essayer : tenter un identifiant et un mot de passe sur un système qui ne lui appartient pas constitue déjà un accès ou une tentative d'accès frauduleux, punissable même sans succès et même avec une intention bienveillante — l'intention « d'aider » n'est pas un titre d'autorisation. La boutique seule, par son propriétaire, peut autoriser un test. Ce qu'il peut faire : signaler factuellement à la boutique ce qu'il a constaté sans rien tester (la page d'administration est exposée publiquement), par un message courtois, et lui conseiller de restreindre l'accès et de consulter un professionnel ; s'il n'obtient pas de réponse, son rôle s'arrête là. Les cinq champs indispensables d'une autorisation : (1) l'identité du commanditaire habilité et sa signature ; (2) le périmètre exact (adresse du site, environnement concerné, exclusions) ; (3) la fenêtre de temps autorisée ; (4) les règles d'engagement et interdictions (pas d'interruption, pas de données réelles) ; (5) les contacts et la procédure d'arrêt d'urgence, ainsi que les livrables attendus.

Leçon 5 — Réponse aux incidents et initiation à l'investigation numérique

Un jour ou l'autre, malgré les protections, un incident survient : poste chiffré par un rançongiciel, site modifié par un inconnu, comptes détournés. La réponse aux incidents est la discipline qui transforme ce mauvais jour en crise maîtrisée. Le guide de référence du NIST (SP 800-61) la découpe en quatre temps. D'abord la préparation, qui se joue bien avant l'incident : un plan écrit, des rôles nommés, une liste de contacts, des sauvegardes régulières et testées, des journaux conservés. Ensuite la détection et l'analyse : signaler, puis qualifier — s'agit-il d'un vrai incident, quelle est son ampleur, quels systèmes et quelles données sont concernés ? Puis le confinement, l'éradication et la récupération : stopper la propagation, supprimer la cause, restaurer proprement à partir de sources saines. Enfin le retour d'expérience : ce qui a manqué, ce qu'on corrige pour la fois suivante. Une organisation sans plan découvre l'incident deux fois : le jour de l'attaque, et le jour du bilan.

Sur le moment, les bons réflexes sont contre-intuitifs, et c'est pourquoi on les prépare. Garder son calme et ouvrir immédiatement un journal de bord horodaté : chaque décision, chaque observation y est consignée, elle servira à tout le monde. Isoler les machines touchées du réseau (débrancher le câble, désactiver le Wi-Fi, bloquer le port au commutateur) pour stopper la propagation — mais sans les éteindre brutalement : la mémoire vive contient des preuves volatiles et, parfois, des clés nécessaires à l'analyse. Ne rien « nettoyer » dans la précipitation : réinstaller ou effacer détruit les traces qui permettent de comprendre l'étendue réelle de l'attaque — a-t-il touché d'autres machines, des données sont-elles parties ? Escalader sans délai : responsable informatique, direction, et si besoin prestataire spécialisé ou centre de réponse aux incidents de rattachement. Communiquer avec mesure, en faits datés, sans accuser ni minimiser. Selon le secteur et les données touchées, des obligations de notification aux autorités ou aux personnes concernées peuvent s'appliquer : c'est un point à vérifier avec la direction et les juristes, pas à improviser.

L'investigation numérique (forensique) commence par un principe : préserver avant d'analyser. On réalise des copies images, bit à bit, des supports concernés, on calcule immédiatement l'empreinte SHA-256 de chaque copie et on la consigne dans un registre de preuves ; tout travail ultérieur se fait sur des copies de ces copies, jamais sur l'original. La chaîne de conservation documente sans trou le parcours de chaque preuve : qui l'a saisie, quand, où elle est stockée, qui y a accédé — sans cette traçabilité, une preuve perd sa valeur, y compris devant un tribunal. L'ordre de collecte suit la volatilité des données : d'abord ce qui disparaît vite (mémoire vive, connexions actives, processus), puis les fichiers temporaires, ensuite les disques et les journaux des serveurs et des équipements réseau. L'analyste reconstitue alors une chronologie minute par minute, distingue ce qui est certain de ce qui est probable, et rédige un rapport factuel : ce qui s'est passé, comment c'est entré, ce qui a été touché, ce qui a été fait. Ce rapport sert la direction, l'assurance, parfois la justice — et toujours le retour d'expérience.

L'essentiel
Réponse aux incidents (NIST SP 800-61) : préparation, détection et analyse, confinement-éradication-récupération, retour d'expérience. Pendant la crise : journal de bord horodaté, isoler sans éteindre, ne rien effacer, escalader. Investigation : copies images avec empreintes SHA-256, chaîne de conservation documentée, collecte des données volatiles d'abord, travail sur copies uniquement.

Exemple — Plan de réponse chronologique (scénario fictif) et registre de preuves

```
SCÉNARIO FICTIF : rançongiciel signalé sur un poste
du secrétariat, lundi 8 h 05 (laboratoire de cours)
============================================================
8 h 05 Détection : l'utilisatrice signale des fichiers
renommés + un message de rançon. Ticket n° 214.
8 h 10 Journal de bord ouvert. Poste ISOLÉ du réseau
(câble débranché), NON éteint : mémoire préservée.
8 h 20 Qualification : incident confirmé. Direction et
responsable informatique alertés. Périmètre à
vérifier : autres postes, serveur de fichiers.
9 h 00 Confinement élargi : compte de la session bloqué,
partage de fichiers suspendu par précaution.
10 h 30 Préservation : copie image du disque réalisée,
empreinte SHA-256 calculée et consignée (voir
registre ci-dessous). Analyse sur la copie.
J+1 Éradication : cause identifiée (pièce jointe
piégée). Postes vérifiés un à un avant retour.
J+1 Récupération : poste réinstallé depuis une image
saine ; données restaurées depuis la sauvegarde
de la veille (testée, hors ligne) ; mots de passe
des comptes concernés renouvelés.
J+3 Retour d'expérience : réunion, rapport final,
mesures décidées (filtrage des pièces jointes,
formation des utilisateurs, sauvegarde revue).

REGISTRE DE PREUVES (extrait)
------------------------------------------------------------
N° 1 : copie image du disque du poste SEC-03
Empreinte SHA-256 :
001c9d792890e0e5efb0b5b3f142e5d2cb64e109ef08f0b0
0411e0cfb13c465a (valeur d'exemple du cours)
Saisie par : technicien de garde, lundi 10 h 30
Stockage : coffre du service, accès journalisé
-> Toute rupture de cette chaîne dévalorise la preuve.
```

À retenir

- Isoler n'est pas éteindre : on coupe le réseau pour stopper la propagation, on garde la machine allumée pour préserver la mémoire et les preuves.
- On n'efface et on ne réinstalle rien avant d'avoir qualifié l'ampleur et préservé les traces : réparer trop tôt, c'est enquêter aveugle.
- Preuve = copie image + empreinte SHA-256 + registre de conservation sans trou ; l'analyse se fait toujours sur des copies.

Exercice

Lundi 7 h 55, avant l'ouverture, le technicien d'astreinte découvre que le site web de l'école affiche une page inconnue à la place de l'accueil (défiguration). Un collègue pressé propose aussitôt de « réinstaller le serveur depuis la sauvegarde de la nuit et ne plus en parler ». Donne la conduite correcte en six actions ordonnées, puis identifie précisément les deux erreurs de la proposition du collègue et ce qu'elles feraient perdre.

✔ Corrigé
Conduite correcte : (1) consigner la découverte dans un journal de bord horodaté et capturer l'état visible (captures d'écran, copie de la page affichée) ; (2) qualifier : le site seul est-il touché, le serveur héberge-t-il d'autres services, des signes d'accès plus profonds existent-ils (comptes créés, journaux d'accès) ; (3) confiner : mettre le site en maintenance ou isoler le service concerné pour ne plus exposer la page piégée au public ; (4) préserver : copie image ou instantané du serveur et sauvegarde des journaux AVANT toute restauration, avec empreintes SHA-256 consignées ; (5) éradiquer et récupérer : identifier la voie d'entrée (composant non mis à jour, compte faible…), la corriger, PUIS restaurer un contenu sain et renouveler les mots de passe des comptes du site ; (6) retour d'expérience et communication factuelle auprès de la direction. Les deux erreurs du collègue : d'abord, restaurer sans préserver détruit les preuves — impossible ensuite de savoir comment l'attaquant est entré, donc rien ne garantit qu'il ne reviendra pas par la même porte le soir même ; ensuite, « ne plus en parler » supprime le retour d'expérience et toute communication maîtrisée : l'incident reste non traité dans sa cause, et si des données ont été touchées, l'organisation manque à ses obligations. Réparer vite est une qualité ; réparer avant de comprendre est une faute professionnelle.

Leçon 6 — Sécurité des réseaux d'entreprise segmentés

Le réseau « plat » est encore le plus répandu dans les petites structures : tous les postes, les serveurs, les imprimantes et les téléphones se voient et peuvent se parler. C'est confortable à installer, et catastrophique le jour d'un incident : un seul poste compromis offre à l'attaquant un boulevard pour explorer tout le reste, rebondir de machine en machine et atteindre les données sensibles — c'est le déplacement latéral. La segmentation inverse la logique : on découpe le réseau en zones de confiance homogènes, selon la sensibilité de ce qu'elles contiennent. Typiquement : une zone pour les postes des utilisateurs, une pour les serveurs internes, une pour les sauvegardes (la plus protégée), une pour l'administration des équipements, une pour les invités et les objets connectés, et une zone démilitarisée (DMZ) qui accueille les services publiés sur Internet. Entre les zones, rien ne passe par défaut : chaque flux traversant doit être explicitement autorisé, justifié et journalisé.

Techniquement, la séparation logique s'appuie sur les VLAN, qui découpent un même équipement réseau en plusieurs réseaux virtuels isolés. Le contrôle des passages se fait sur un pare-feu inter-zones (ou sur les fonctions de filtrage des commutateurs/routeurs) avec une politique claire : tout refuser par défaut, puis ouvrir au cas par cas — quelle zone source, quelle zone destination, quel service, dans quel sens. Par exemple, les utilisateurs peuvent consulter le serveur web interne en HTTPS, mais seul le serveur web de la DMZ peut interroger la base de données, et uniquement sur son port de service ; les sauvegardes ne sont joignables que par le serveur de sauvegarde, jamais depuis les postes. L'administration des équipements réseau et des serveurs se fait depuis la zone d'administration, idéalement via un poste bastion dédié et authentifié, jamais depuis le Wi-Fi invités. Les journaux des flux refusés et des accès sensibles sont conservés : ils alimentent la détection étudiée à la leçon 5. Un point

souvent oublié : la DMZ n'est pas un fourre-tout — un serveur publié compromis ne doit ouvrir aucune porte directe vers le cœur du réseau.

La segmentation réussie est un travail d'entretien, pas un événement. Il faut un inventaire des flux légitimes (qui a besoin de parler à quoi, et pourquoi), revu régulièrement : les règles « temporaires » oubliées et les ouvertures « any vers any » sont les failles classiques des matrices vieillissantes. Les équipements réseau eux-mêmes reçoivent leurs correctifs et leurs mots de passe d'administration robustes et uniques. On vérifie le cloisonnement par des contrôles défensifs réguliers : depuis un poste de la zone invités, seul l'Internet doit être visible — jamais un serveur interne. La démarche moderne dite « zéro confiance » prolonge cette logique : ne plus accorder d'accès du seul fait que l'on se trouve « à l'intérieur » du réseau, mais authentifier et autoriser chaque accès, d'où qu'il vienne. Même à petite échelle — une école, une PME — quatre ou cinq zones bien tenues changent radicalement la portée d'un incident : ce qui aurait été une catastrophe devient un poste à réinstaller.

L'essentiel
Segmenter, c'est découper le réseau plat en zones de confiance (utilisateurs, serveurs, sauvegardes, administration, invités, DMZ) reliées par un pare-feu en « tout refuser par défaut », avec des flux explicites, minimaux et journalisés. Résultat : un incident reste local au lieu de devenir général.

Exemple — Schéma de segmentation d'une petite entreprise et matrice des flux autorisés

```
SCHÉMA (vue simplifiée)
============================================================
INTERNET
|
[ PARE-FEU ]
|
+---------+----------+
| DMZ | services publiés
| - serveur web | (site vitrine)
| - relais courriel |
+---------+----------+
| flux filtrés
[ PARE-FEU INTERNE ]
+------------+--------+---------+----------------+
| | | |
ZONE POSTES ZONE SERVEURS ZONE SAUVEGARDES ZONE ADMIN
VLAN 10 VLAN 20 VLAN 30 VLAN 99
employés appli + base serveur de bastion
sauvegarde d'admin.
+--- ZONE INVITÉS / Wi-Fi visiteurs : VLAN 40,
accès Internet seul, AUCUN accès interne

MATRICE DES FLUX AUTORISÉS (tout le reste est REFUSÉ)
------------------------------------------------------------
Source -> Destination Service Pourquoi
Internet -> DMZ web 443 site public
DMZ web -> SERVEURS base 5432 l'appli lit ses
données, et rien
d'autre
POSTES -> SERVEURS appli 443 travail quotidien
SERVEURS -> SAUVEGARDES 10000 sauvegarde noct.
ADMIN bastion -> tous équipements SSH/8443 administration
INVITÉS -> Internet seul - aucun accès int.
-> Chaque ligne se justifie ; chaque refus est journalisé.
```

À retenir

- Par défaut, tout flux entre zones est refusé ; on n'ouvre que le nécessaire, dans un seul sens, pour un service précis.
- Les sauvegardes vivent dans la zone la plus fermée : c'est elles que les rançongiciels cherchent en premier.
- Une matrice de flux se relit et se nettoie : une règle devenue inutile est une porte qu'on a oublié de refermer.

Exercice

Voici quatre règles trouvées dans la matrice d'un pare-feu interne : (R1) INVITÉS -> SERVEURS DE FICHIERS, tous ports, autorisé « pour dépanner les visiteurs » ; (R2) POSTES -> SAUVEGARDES, port 10000, autorisé en permanence ; (R3) DMZ web -> SERVEURS base de données, port 5432, autorisé ; (R4) TOUTES ZONES -> TOUTES ZONES, tous ports, autorisé, ajouté « temporairement » il y a deux ans. Pour chaque règle : verdict (conserver, restreindre, supprimer) et justification en une phrase.

✔ Corrigé
R1 : supprimer — la zone invités ne doit jamais atteindre les serveurs de fichiers ; le dépannage des visiteurs se fait autrement (accès Internet seul), et « tous ports » est de toute façon inacceptable. R2 : restreindre — les postes n'ont aucune raison de joindre directement les sauvegardes ; seules les machines autorisées (le serveur de sauvegarde et les serveurs à sauvegarder) doivent parler au système de sauvegarde, ce qui protège les copies contre les rançongiciels partis d'un poste. R3 : conserver — c'est le flux légitime de l'application publiée : source unique (le serveur web de la DMZ), destination unique (la base), port unique (5432) ; il doit rester journalisé et la base ne doit accepter de connexion que depuis ce serveur. R4 : supprimer immédiatement et auditer — une règle « tout vers tout » annule toute la segmentation ; son ancienneté montre exactement le défaut d'entretien décrit dans la leçon : après suppression, on vérifie quels flux légitimes éventuels doivent être ré-ouverts proprement, un par un et justifiés.

Leçon 7 — Projet : audit de sécurité d'un site de test autorisé

Ce projet rassemble tout le cours dans une démarche complète, et il commence — comme tout travail de sécurité professionnel — par le cadre. Chaque équipe construit son propre petit site web de test (par exemple une mini-bibliothèque ou un catalogue de produits, avec des données entièrement fictives), hébergé sur une machine virtuelle ou un poste du laboratoire, OU reçoit l'environnement de test fourni par l'enseignant. Avant la moindre vérification, l'équipe rédige son autorisation écrite : l'enseignant et les membres de l'équipe la signent, le périmètre y est figé (l'adresse locale du seul site de test), les règles d'engagement y figurent (vérifications non destructives uniquement, aucune donnée réelle, aucun autre système visé — ni le réseau de l'école, ni un site extérieur, jamais). Ce document n'est pas une formalité : c'est lui qui rend le travail légal et professionnel, exactement comme dans la leçon 4.

La conduite de l'audit mobilise les leçons précédentes dans l'ordre. D'abord l'inventaire : pages et formulaires du site, comptes et rôles, versions de chaque composant, services réellement actifs. Ensuite la revue défensive, point par point : le site force-t-il HTTPS avec un certificat cohérent pour le laboratoire (leçon 2) ? Les en-têtes de sécurité sont-ils présents ? Les cookies de session portent-ils les protections attendues (HttpOnly, SameSite, Secure dès qu'il y a HTTPS) ? Les mots de passe des comptes de test sont-ils stockés en hachages lents et salés (leçon 3) ? Les requêtes sont-elles toutes paramétrées, les affichages encodés ? Le contrôle d'accès est-il vérifié côté serveur pour chaque rôle, avec les comptes de test de l'équipe ? Les journaux enregistrent-ils les connexions et les erreurs, et existe-t-il une sauvegarde du site, restaurable, conservée à part ? Chaque vérification se fait avec des gestes non destructifs sur son propre environnement : navigateur et ses outils de développement, examen des en-têtes de réponse, lecture de son propre code et de sa propre configuration. Chaque constat est documenté au fur et à mesure : preuve (capture, extrait de configuration), catégorie OWASP correspondante, risque expliqué simplement, correctif proposé.

Le livrable est un rapport d'audit professionnel, en deux parties : une synthèse d'une page pour un décideur non technicien (état général, trois risques majeurs, décisions demandées), puis le détail des constats classés par criticité — critique, élevée, moyenne, faible — chaque niveau étant justifié par l'impact et la facilité de constat dans l'environnement de test. Le rapport se termine par un plan de remédiation priorisé et daté. L'équipe applique ensuite les correctifs des constats critiques et élevés sur son site, puis procède à la contre-vérification : chaque point corrigé est re-testé et marqué « confirmé corrigé » ou « persiste ». La soutenance présente la démarche et démontre les corrections — jamais une exhibition d'attaques. L'évaluation porte sur la rigueur du cadre (autorisation, périmètre), l'exhaustivité de la revue, la qualité des correctifs et la clarté du rapport : les compétences exactes d'un auditeur junior.

L'essentiel
Le projet applique la démarche complète sur le site de test de l'équipe : autorisation écrite et périmètre figé, inventaire, revue défensive guidée par les leçons 1 à 6, constats documentés et classés, plan de remédiation, corrections, contre-vérification, rapport en deux parties (décideurs + technique). On démontre des correctifs, pas des attaques.

Exemple — Grille d'audit remplie (extrait) pour le site fictif « BiblioTest » du laboratoire

```
GRILLE D'AUDIT — Site de test « BiblioTest » (équipe 4)
Environnement : machine virtuelle du labo, 192.168.56.20
Autorisation : signée le 03/10 (enseignant + équipe)
============================================================
N° Point vérifié État Catégorie
-- ------------------------------ -------- ----------
1 HTTPS actif et forcé CONFORME TLS
2 Certificat adapté au labo CONFORME TLS
(auto-signé, empreinte connue, usage interne seul)
3 En-têtes de sécurité présents À CORR. OWASP A05
(manque X-Content-Type-Options et le cadre anti-
intégration frauduleuse des pages)
4 Cookies de session protégés À CORR. OWASP A07
(HttpOnly présent, SameSite absent)
5 Mots de passe : hachage salé CONFORME OWASP A02
et lent (bcrypt vérifié dans le code)
6 Requêtes SQL paramétrées CONFORME OWASP A03
(revue de code complète, aucun collage de saisie)
7 Affichages encodés (anti-XSS) CONFORME OWASP A03
8 Contrôle d'accès par rôle À CORR. OWASP A01
(la page « gestion » est protégée par le seul menu
caché ; le serveur ne revérifie pas le rôle)
9 Composants à jour CONFORME OWASP A06
10 Journalisation connexions/erreurs CONFORME OWASP A09
11 Sauvegarde testée et séparée CONFORME Gestion
(restauration essayée le 05/10 sur la machine de test)
------------------------------------------------------------
Synthèse : 8 conformes / 3 à corriger ; le constat n° 8
est classé ÉLEVÉ (contournement du rôle par simple
adresse), les n° 3 et 4 MOYENS. Plan : corriger le n° 8
cette semaine, les autres avant la soutenance.
```

À retenir

- Pas d'audit sans autorisation écrite, même sur le site de sa propre équipe : le réflexe documentaire fait partie du métier.
- Un constat utile = preuve + catégorie + risque expliqué + correctif proposé + criticité justifiée.
- L'audit se termine par des corrections et une contre-vérification, pas par le rapport : la sécurité livrée vaut mieux que la sécurité racontée.

Exercice

À partir de la grille d'audit de l'exemple (constats n° 3, 4 et 8 « à corriger »), rédige la partie « synthèse pour décideurs » du rapport : cinq à huit lignes maximum, sans jargon inutile, qui donnent l'état général, les risques majeurs dans l'ordre, et les décisions ou moyens demandés. Puis justifie en deux phrases pourquoi le constat n° 8 passe avant les deux autres dans le plan de remédiation.

✔ Corrigé
Exemple de synthèse attendue : « L'audit du site de test de la bibliothèque, réalisé dans le cadre autorisé du laboratoire, donne un état général satisfaisant : huit contrôles sur onze sont conformes, dont le stockage protégé des mots de passe et des sauvegardes testées. Un risque élevé subsiste : la page de gestion interne peut être atteinte par un compte ordinaire, car le serveur ne revérifie pas le rôle de l'utilisateur ; les relevés pourraient être consultés ou modifiés par des personnes non habilitées. Deux risques moyens concernent des protections de session et d'affichage incomplètes. Nous demandons la validation du plan suivant : correction du contrôle des rôles sous une semaine par l'équipe, protections complémentaires avant la soutenance, puis contre-vérification par l'auditeur. Aucune donnée réelle n'est concernée, l'audit portant sur un environnement de test. » Justification de la priorité : le constat n° 8 touche au contrôle d'accès (OWASP A01), c'est-à-dire à la confidentialité et à l'intégrité des données de l'application, avec un contournement à la portée de n'importe quel compte ordinaire ; les constats n° 3 et n° 4 durcissent des protections utiles mais face à des scénarios plus conditionnels — l'impact et la simplicité du n° 8 imposent donc son traitement en premier.

Travaux dirigés (TD)

TD 1 — Lire un certificat X.509 et juger trois alertes

On te présente la fiche d'un certificat reçu par un navigateur : Sujet (SAN) : boutique.iam-edu.example ; Émetteur : AC Intermédiaire Enseignement ; Clé publique : RSA 2 048 bits ; Validité : du 1er septembre au 30 novembre ; Algorithme de signature : SHA-256 avec RSA ; la chaîne remonte à « AC Racine Enseignement », présente dans le magasin de confiance du poste. Questions : 1) Pour chaque champ, dis ce qu'il garantit. 2) Le navigateur accepte-t-il ce certificat le 15 octobre pour le site boutique.iam-edu.example ? Justifie point par point. 3) Juge trois variantes : (a) même certificat présenté le 15 décembre ; (b) même certificat présenté par le site copie.iam-edu.example ; (c) certificat identique mais auto-signé par le serveur lui-même, sur le site public de la boutique. Pour chacune : acceptée ou alerte, quelle vérification décide, et quelle est la bonne réaction de l'utilisateur ?

✔ Corrigé du TD 1
1) Le SAN désigne les noms de domaine couverts : le certificat ne vaut que pour eux. L'émetteur indique quelle autorité a signé et donc à qui l'on fait confiance par délégation. La clé publique RSA 2 048 bits est la clé du site, au niveau de sécurité actuel. Les dates bornent la validité dans le temps. L'algorithme de signature SHA-256 avec RSA indique comment l'AC a signé : un algorithme encore considéré comme sûr. La racine présente dans le magasin du poste fournit le point d'arrêt de la chaîne : sans elle, impossible de conclure. 2) Oui, acceptation le 15 octobre : chaîne complète et signatures vérifiées jusqu'à une racine de confiance ; nom demandé = nom du SAN ; date dans la période de validité ; (on supposera la non-révocation vérifiée). 3) (a) Alerte « certificat expiré » : la vérification des dates échoue ; réaction : ne pas poursuivre sur un site réel, signaler au service informatique qui doit renouveler — c'est le scénario classique du renouvellement oublié. (b) Alerte « nom incorrect » : le SAN ne contient pas le nom visité ; sur un site « copie », c'est un signal d'hameçonnage : fermer, ne rien saisir, signaler. (c) Alerte « autorité inconnue » : auto-signé signifie que personne de confiance ne garantit l'identité — acceptable uniquement en laboratoire fermé avec empreinte communiquée par l'enseignant, jamais pour un site public de boutique : à refuser et à faire corriger par un certificat d'une AC reconnue.

TD 2 — Auditer et corriger une matrice de flux réseau

Une école a segmenté son réseau en quatre zones : POSTES (enseignants et étudiants), SERVEURS (application de notes + base), SAUVEGARDES, INVITÉS (Wi-Fi visiteurs). La matrice actuelle contient les règles suivantes : (M1) POSTES -> SERVEURS application, port 443 : autorisé. (M2) POSTES -> SAUVEGARDES, tous ports : autorisé « au cas où ». (M3) INVITÉS -> SERVEURS base de données, port 5432 : autorisé « pour les démonstrations ». (M4) SERVEURS -> SAUVEGARDES, port 10 000, de 22 h à 5 h : autorisé. (M5) SERVEURS base -> INTERNET, tous ports : autorisé. (M6) Aucune journalisation des refus n'est activée. Travail : pour chaque ligne, verdict (conserver / restreindre / supprimer) motivé ; propose ensuite la matrice corrigée minimale, et cite deux flux légitimes manquants qu'il faudra peut-être ajouter en les justifiant.

✔ Corrigé du TD 2
M1 : conserver — flux de travail normal, source, destination et port précis. M2 : supprimer — les postes ne doivent jamais joindre directement les sauvegardes ; seuls les serveurs à sauvegarder et le serveur de sauvegarde ont à y accéder ; « au cas où » et « tous ports » sont les deux formulations à bannir d'une matrice. M3 : supprimer — la zone invités ne doit atteindre aucun système interne, et surtout pas la base de données ; les démonstrations se font sur un environnement de test séparé, pas sur la production. M4 : conserver — sauvegarde nocturne planifiée, flux unique, port unique, fenêtre horaire limitée : c'est le modèle d'une bonne règle. M5 : restreindre — la base de données n'a pas à naviguer librement sur Internet ; seuls des flux précis et justifiés (par exemple les mises à jour du système, via un relais contrôlé) peuvent être ouverts, sinon le serveur devient une porte de sortie pour des données volées. M6 : corriger en activant la journalisation des refus et des accès sensibles, conservée et consultée : sans journaux, pas de détection (leçon 5). Matrice corrigée minimale : M1, M4, plus les règles par défaut « tout refuser » entre zones et la journalisation. Deux flux légitimes souvent manquants : POSTES -> serveur d'impression ou de fichiers internes (service précis et limité) si l'école l'utilise, et ADMIN (bastion) -> équipements réseau en SSH/HTTPS d'administration ; chacun doit être ajouté seulement s'il est réel, avec source, destination, port et justification écrits.

Travaux pratiques (TP)

TP 1 — Durcir ton site de test local et prouver le durcissement

Objectif : appliquer les réglages défensifs du cours à TON site de test du laboratoire (jamais à un site tiers ni au réseau de production) et produire la preuve avant/après. Étape 1 — Relevé initial : avec les outils de développement du navigateur (onglet Réseau) ou une simple demande en ligne de commande vers ton adresse locale, note les en-têtes de réponse actuels de ton site et l'état de ses cookies de session ; consigne le tout dans un tableau « avant ». Étape 2 — HTTPS de laboratoire : active HTTPS sur ton serveur local avec un certificat auto-signé réservé au laboratoire ; note l'empreinte du certificat et explique en deux phrases pourquoi l'alerte du navigateur est ici attendue et acceptable, et pourquoi elle ne le serait pas en public. Étape 3 — En-têtes et cookies : ajoute les en-têtes de protection (interdiction du sniffing de type, protection contre l'intégration de tes pages dans des cadres tiers, politique de référent sobre) et passe tes cookies de session en HttpOnly + SameSite, avec Secure puisque HTTPS est actif. Étape 4 — Mots de passe : vérifie dans ta base de test qu'aucun mot de passe n'est lisible et que le stockage utilise un hachage lent et salé ; sinon, corrige le code d'inscription de ton site de test. Étape 5 — Re-relevé : refais l'étape 1, complète le tableau « après », et écris cinq lignes de conclusion : ce qui a changé, ce qui reste à faire, ce que tu as appris.

✔ Attendus du TP 1
Livrables attendus : (1) un tableau avant/après complet — pour chaque en-tête et chaque attribut de cookie : absent avant, présent après, ou l'inverse justifié ; (2) l'empreinte du certificat de laboratoire et le paragraphe expliquant le statut de l'auto-signé (confiance manuelle et limitée au laboratoire, empreinte communiquée par un canal sûr) ; (3) la capture ou l'extrait montrant qu'en base, les mots de passe sont des hachages salés (on ne doit voir ni texte clair, ni hachages rapides identiques pour deux comptes ayant le même mot de passe) ; (4) la conclusion en cinq lignes. Points de vigilance pour l'évaluation : le travail ne concerne que l'environnement local de l'étudiant ; un en-tête annoncé mais absent du relevé « après » invalide la preuve — c'est le relevé qui fait foi, pas l'intention ; l'attribut Secure n'a de sens qu'avec HTTPS ; SameSite et HttpOnly doivent être visibles dans le cookie réellement émis par le serveur de test.

Exercices corrigés supplémentaires

Exercice 1

Choix de paramètres cryptographiques : trois propositions circulent dans un ancien tutoriel que tu dois évaluer. (a) « Pour ton nouveau serveur, RSA 1 024 bits suffit largement et c'est plus rapide. » (b) « Pour archiver les dossiers des étudiants pour dix ans, prends AES-256 plutôt qu'AES-128. » (c) « Pour vérifier que le logiciel téléchargé est intègre, compare son empreinte SHA-1 publiée par l'éditeur. » Pour chacune : acceptée ou refusée, et pourquoi en deux phrases maximum.

✔ Corrigé
(a) Refusée : RSA 1 024 bits est considéré comme insuffisant face aux moyens de calcul actuels et aux recommandations en vigueur (ANSSI, NIST) ; le minimum est 2 048 bits, et le gain de vitesse ne justifie pas un risque sur la clé maîtresse du service. (b) Acceptée : les deux tailles sont sûres aujourd'hui, mais AES-256 offre une marge supplémentaire précieuse pour des archives qui doivent rester confidentielles dix ans, à un coût de calcul modeste — c'est le choix prudent pour le long terme. (c) Refusée : des collisions SHA-1 ont été démontrées publiquement (deux fichiers différents, même empreinte), donc SHA-1 ne prouve plus l'intégrité ; on utilise SHA-256 ou plus récent, et lorsque l'éditeur le propose, on vérifie aussi la signature du fichier.

Exercice 2

Ton cousin gère le site de sa boutique. Au téléphone, il te demande : « Passe ce week-end, vérifie si mon site est solide, je te fais entièrement confiance. » Il n'a rien écrit, l'hébergement est mutualisé avec d'autres sites, et il te propose de travailler directement sur le site en ligne. Que dois-tu exiger ou corriger avant de toucher à quoi que ce soit ? Rédige ta réponse en distinguant : ce qui doit être écrit, ce qui doit changer dans les conditions techniques, et ce qui resterait interdit même avec son accord.

✔ Corrigé
Ce qui doit être écrit : une autorisation signée du cousin en tant que propriétaire du site, précisant le périmètre (son site uniquement, nommé explicitement), la fenêtre de test, les règles d'engagement (vérifications non destructives, pas de données clients réelles manipulées), les contacts et la procédure d'arrêt. Une confiance orale, même sincère, ne protège ni lui ni toi en cas de problème. Ce qui doit changer techniquement : travailler sur une copie ou un environnement de test, pas directement sur le site en production ; si l'hébergeur mutualisé impose des règles (beaucoup interdisent tout test sans leur propre accord), les respecter — l'autorisation du cousin ne couvre pas l'infrastructure de l'hébergeur ni les sites voisins, qui sont hors périmètre par définition. Ce qui resterait interdit même avec son accord : tester les autres sites du même serveur, interrompre le service pendant les heures d'ouverture, copier ou consulter la base des clients réels au-delà du strict nécessaire à un constat, et bien sûr tout système qui n'est pas le sien. S'il refuse l'écrit, la réponse professionnelle est de décliner le test et de se limiter à des conseils généraux de durcissement (mises à jour, mots de passe, sauvegardes), qui ne demandent aucun accès intrusif.

Exercice 3

L'éditeur d'un logiciel annonce sur sa page officielle l'empreinte SHA-256 de son installeur : 9f2c…a41b (valeur abrégée pour l'exercice). Après téléchargement, ton calcul donne une empreinte différente. Un camarade te dit : « C'est sûrement juste le réseau qui a modifié un détail, installe quand même, tu gagneras du temps. » Explique pourquoi l'écart compte, la conduite correcte étape par étape, et le seul cas où une différence d'empreinte peut être légitime.

✔ Corrigé
L'écart compte parce qu'une fonction de hachage cryptographique comme SHA-256 a un effet avalanche : modifier un seul bit du fichier change environ la moitié des bits de l'empreinte. Une empreinte différente signifie donc un fichier différent — corrompu, tronqué, ou remplacé par une version piégée ; le « détail modifié par le réseau » du camarade n'existe pas en pratique, les protocoles de transfert détectent et corrigent les erreurs ordinaires. Conduite correcte : ne pas installer ; supprimer le fichier suspect ; re-télécharger depuis la source officielle (vérifier que l'adresse est bien celle de l'éditeur, en HTTPS) ; recalculer l'empreinte et la comparer caractère par caractère à la valeur officielle complète, pas à une valeur recopiée d'un forum ; si l'éditeur fournit en plus une signature, la vérifier ; si l'écart persiste, signaler le problème à l'éditeur par son canal officiel et attendre. Le seul cas légitime de différence : l'empreinte de référence ne correspond pas à cette version du fichier (autre système, autre édition, ancienne version) — ce qui se tranche en retournant à la page officielle de la version exacte téléchargée, jamais en installant « quand même ».

QCM d'évaluation

Choisis la bonne réponse pour chaque question. Les corrigés détaillés se trouvent juste après le QCM : ne les regarde qu'après avoir répondu à tout.

1. Quelle est la taille d'un bloc chiffré par AES, quelle que soit la clé utilisée ?

A) 64 bits

B) 128 bits

C) 256 bits

2. Dans le chiffrement hybride utilisé par TLS, quel est le rôle de RSA ?

A) Chiffrer la totalité des données échangées

B) Remplacer les certificats

C) Protéger le transport de la clé de session et signer

3. La confidentialité persistante (perfect forward secrecy) de TLS 1.3 vient…

A) De l'échange de clés éphémères (ECDHE), renouvelé à chaque session

B) Du certificat du serveur conservé dix ans

C) De la longueur du mot de passe de l'utilisateur

4. OWASP Top 10 (2021), catégorie A03 — Injection : le correctif fondateur est…

A) Interdire le caractère apostrophe dans les formulaires

B) Utiliser des requêtes paramétrées et encoder les sorties

C) Masquer les formulaires aux utilisateurs non connectés

5. Quelle est la condition qui rend un test d'intrusion légal ?

A) L'accord verbal de l'utilisateur du poste testé

B) Le fait de ne causer aucun dégât

C) Une autorisation écrite préalable du propriétaire du système, avec un périmètre défini

6. Lors d'un incident, la première action technique sur une machine suspecte est de…

A) L'isoler du réseau sans l'éteindre brutalement

B) La réinstaller immédiatement depuis la sauvegarde

C) L'éteindre et retirer son disque pour le formater

7. Pourquoi calcule-t-on l'empreinte SHA-256 d'une copie de disque saisie comme preuve ?

A) Pour compresser la copie

B) Pour prouver que la copie n'a pas été modifiée depuis sa saisie

C) Pour chiffrer le contenu du disque

8. Dans un réseau segmenté, la règle de base entre les zones est…

A) Tout autoriser, sauf ce qui est manifestement dangereux

B) Autoriser ce que demandent les utilisateurs

C) Tout refuser par défaut, puis autoriser explicitement les flux nécessaires

9. Que vérifie le navigateur quand il reçoit le certificat d'un site ?

A) Uniquement que le site répond vite

B) La signature et la chaîne jusqu'à une racine de confiance, le nom de domaine et les dates de validité

C) Que le site possède un logo officiel

10. Comment doit-on stocker les mots de passe des utilisateurs d'une application ?

A) Chiffrés avec AES pour pouvoir les restituer en cas d'oubli

B) Hachés avec une fonction lente et salée conçue pour cela (Argon2, bcrypt)

C) En texte clair, dans une table protégée par un mot de passe

Corrigés du QCM

1. Réponse B — AES travaille toujours sur des blocs de 128 bits ; ce sont les clés qui varient : 128, 192 ou 256 bits.

2. Réponse C — RSA, trop lent pour les volumes, protège la petite clé symétrique de session et sert aux signatures ; les données sont chiffrées en AES.

3. Réponse A — Chaque session fabrique un secret neuf et éphémère : voler plus tard la clé privée du serveur ne rouvre pas les anciennes conversations.

4. Réponse B — La paramétrisation sépare strictement le code des données ; l'encodage des sorties neutralise les scripts injectés. Filtrer un caractère est toujours contournable.

5. Réponse C — Sans autorisation écrite du propriétaire, tester un système tiers est un accès frauduleux — même sans dégât et même avec de bonnes intentions.

6. Réponse A — Isoler stoppe la propagation ; garder la machine allumée préserve la mémoire vive et les preuves ; réinstaller trop tôt détruit les traces et masque l'ampleur réelle.

7. Réponse B — L'empreinte, consignée au registre avec la chaîne de conservation, permet de démontrer l'intégrité de la preuve à tout moment.

8. Réponse C — Le moindre privilège réseau : chaque flux ouvert est nommé, minimal, justifié et journalisé ; tout le reste est refusé.

9. Réponse B — Chaîne de signatures, correspondance du nom (SAN), période de validité et non-révocation : un seul échec déclenche l'alerte.

10. Réponse B — Le hachage lent et salé est irréversible et coûteux à attaquer en masse ; la restitution d'un mot de passe oublié passe par une réinitialisation, jamais par un renvoi du mot de passe.

Résumé — les notions importantes

• Cryptographie : AES (blocs de 128 bits, clés de 128/192/256 bits, mode authentifié GCM) chiffre les données, RSA (2 048 bits minimum) transporte les clés et signe, SHA-256 fournit les empreintes d'intégrité ; le certificat numérique lie une identité à une clé publique.

• PKI et TLS : chaîne de confiance (racine, intermédiaires, certificat X.509), cycle de vie administré (demande, validation, renouvellement, révocation) ; TLS 1.3 authentifie le serveur et crée un secret de session éphémère — la confidentialité persistante.

• OWASP Top 10 (2021) : dix familles de risques web et leurs correctifs côté défenseur — contrôle d'accès côté serveur, requêtes paramétrées, encodage des sorties, hachage lent et salé des mots de passe, composants à jour, journalisation.

• Tests d'intrusion : jamais sans autorisation écrite préalable, périmètre précis et règles d'engagement ; méthodologie en six phases, validation non destructive, produit = un rapport de correctifs protégé.

• Réponse aux incidents (NIST SP 800-61) : préparation, détection et analyse, confinement-éradication-récupération, retour d'expérience ; pendant la crise : journal horodaté, isoler sans éteindre, ne rien effacer, escalader.

• Investigation numérique : préserver avant d'analyser — copies images, empreintes SHA-256, chaîne de conservation documentée, données volatiles collectées en premier, travail uniquement sur des copies.

• Segmentation réseau : zones de confiance reliées par un pare-feu en « tout refuser par défaut » ; flux explicites, minimaux et journalisés ; sauvegardes dans la zone la plus fermée ; l'audit du projet final applique l'ensemble, sur un site de test autorisé.

Vidéos associées aux leçons

Ces vidéos, en français, complètent les leçons. Liens vérifiés au moment de la publication de ce document.

Leçon 1 — Le chiffrement symétrique - AES expliqué

Chaîne : Mickaël Dupont · Langue : Français · Niveau : Intermédiaire

Le fonctionnement interne d'AES expliqué pas à pas : blocs, substitutions, permutations et tours de chiffrement. Le complément visuel idéal de la première partie de la leçon 1, pour comprendre ce que fait réellement l'algorithme derrière la démonstration Python.

https://www.youtube.com/watch?v=5ZEYKk8BHcE

Leçon 1 — Comprendre la cryptographie par la pratique avec OpenSSL

Chaîne : alumbo · Langue : Français · Niveau : Intermédiaire

La cryptographie asymétrique en pratique : génération d'une paire de clés RSA, chiffrement avec la clé publique et déchiffrement avec la clé privée, en ligne de commande. Elle prolonge la démonstration de chiffrement hybride de la leçon 1 et prépare la leçon 2 sur les certificats.

https://www.youtube.com/watch?v=MdVpxNHDXtc

Leçon 2 — Comprendre le chiffrement SSL / TLS avec des emojis (et le HTTPS)

Chaîne : Cookie connecté · Langue : Français · Niveau : Débutant à intermédiaire

Une explication très visuelle de ce qui se passe derrière une connexion HTTPS : rôle du certificat, échange de clés et passage au chiffrement symétrique. Parfaite pour fixer la poignée de main TLS et la chaîne de confiance de la leçon 2 avant d'entrer dans les détails.

https://www.youtube.com/watch?v=7W7WPMX7arI

Leçon 5 — NIS2 : notification d'incident, fiche réflexe et qualification (ReCyF 12/20)

Chaîne : SECURI GEEK · Langue : Français · Niveau : Intermédiaire à avancé
Comment détecter, qualifier et notifier un incident de sécurité sans perdre de temps, avec une fiche réflexe d'une page pensée pour être lue sous stress. Un prolongement très concret de la leçon 5, côté obligations et organisation de la réponse.

https://www.youtube.com/watch?v=zxxzmkdCBGY

Leçon 5 — Cybersécurité en zone OHADA : maîtriser la gestion des incidents de sécurité du SI

Chaîne : MICHEL MARC MATIP · Langue : Français · Niveau : Intermédiaire
La gestion des incidents de sécurité adaptée au contexte juridique et organisationnel de l'espace OHADA, dont fait partie le Mali : étapes clés, bonnes pratiques et erreurs à éviter. Le regard « terrain » qui complète la méthode NIST de la leçon 5.

https://www.youtube.com/watch?v=eo9DlGQFKcs

Sources et références

• ANSSI (Agence nationale de la sécurité des systèmes d'information) — « Guide d'hygiène informatique » et guides de réponse aux incidents, publications officielles en accès libre sur anssi.gouv.fr.

• OWASP Foundation — « OWASP Top 10 » (édition 2021, référence de ce cours) et « Application Security Verification Standard (ASVS) », ressources gratuites sur owasp.org.

• NIST — SP 800-61 Rev. 2, « Computer Security Incident Handling Guide » ; FIPS 197, « Advanced Encryption Standard (AES) » : les documents de référence des leçons 1 et 5.

• IETF — RFC 8446, « The Transport Layer Security (TLS) Protocol Version 1.3 » : le texte officiel du protocole étudié à la leçon 2.

• Programme officiel Licence Informatique, Université IAM (Institut Africain de Management), Bamako — vérifier les intitulés d'UE auprès du secrétariat pédagogique.

• Les vidéos listées dans la section « Vidéos associées aux leçons » de ce document (liens vérifiés à la publication).

Fin du cours — Bronbah Tech, Bibliothèque Universitaire IAM. Bon courage pour tes révisions !